

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-ad3ca34b928239eca.jsonl`  
- SHA-256: `7dd368b38f8d22d135b53563bc067e24092e1c745c640178de932dec04a8e6be`  
- Source modified: `2026-07-06T06:10:18+00:00`  
- Imported at: `2026-07-08T16:00:13+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:08:32.723Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db._apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension schema_sql):

title: Backfill INNER JOIN silently drops orphan videos, making them re-deliverable duplicates
severity(claimed): medium
file:line: src/tg_video_filter/db.py:562
summary: Migration 003's content_items backfill uses an INNER JOIN sources s ON s.provider_source_id = CAST(v.channel_id AS TEXT); any legacy videos row whose channel_id has no

matching channels row is silently discarded, so the migrated app no longer sees it as seen and will re-ingest and re-forward it.

failure_scenario: In v1 the channel upsert in telethon_client.py:240-247 is explicitly best-effort and non-fatal (exception caught + logged), while insert_video at line 263 proceeds regardless, and videos.channel_id has NO foreign key to channels (git show main:db.py). So a v1 DB can legitimately hold a videos row (say channel_id=-1005, message_id=77, document_id=888, matched=1, forwarded=1) with no channels row for -1005. Running init_db -> migration 003 backfills 0 content_items for it (JOIN finds no source). Repro (scratchpad/repro8.py): videos_deprecated rows=1, content_items rows=0; db.get_video(1,-1005,77) -> None and get_video_by_document_id(1,888) -> None. Post-migration the video is invisible to every dedup shim, so on next scan it is treated as new and forwarded again -> duplicate delivery to the user, and its matched/forwarded state is lost.

evidence: src/tg_video_filter/db.py:562-566 (INNER JOIN sources s ... AND s.provider_source_id = CAST(v.channel_id AS TEXT)); telethon_client.py:238-247 (channel upsert best-effort, failures non-fatal) with insert_video at 263 proceeding anyway; git show main:src/tg_video_filter/db.py videos DDL has no REFERENCES channels. Reproduced end-to-end in scratchpad/repro8.py: orphan video -> content_items=0, both dedup shims return None ("INVISIBLE post-migration (will be re-delivered)").

recommendation: Either use a LEFT JOIN and synthesize/backfill the missing source row (e.g. INSERT the channel into sources first from DISTINCT videos.channel_id, not only from channels), or add a post-backfill assertion that SELECT COUNT(*) FROM content_items equals COUNT over the source videos and fail the migration loudly if rows were dropped. A migration that silently loses rows should not be INSERT OR IGNORE against an INNER JOIN without a reconciliation check.

2. assistant (2026-07-06T06:08:37.948Z)

I'll verify this finding by reading the actual code at the cited lines and tracing the control flow.

3. user (2026-07-06T06:08:39.019Z)

```
500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     #     channels stay watched (watched=1). provider_source_id is the Telegram
516     #     channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521              seen_first_at, last_seen_at)
522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524         FROM channels
525         """
526     )
527
528     # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529     #     The three-branch decision tree (Option 1A):
530     #     a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531     #     b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532     #     c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533     #     The ``CASE`` is ordered strongest-first and each branch is guarded so
```

```

534 # partial metadata never collapses to a degenerate key. source_id is
535 # resolved by joining the channel's provider_source_id back to sources.id
536 # (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540         (account_id, source_id, provider, provider_message_id, content_type,
541          document_id, duration_s, width, height, size_bytes, mime_type,
542          dedup_key, matched, seen_at)
543     SELECT
544         v.user_id,
545         s.id,
546         'telegram',
547         CAST(v.message_id AS TEXT),
548         'video',
549         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551         CASE
552             WHEN v.document_id IS NOT NULL
553             THEN 'tg:doc:' || v.document_id
554             WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555                  AND v.height IS NOT NULL AND v.width IS NOT NULL
556             THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557                  || v.height || ':' || v.width
558             ELSE 'tg:msg:' || s.id || ':' || v.message_id
559         END,
560         v.matched,
561         v.seen_at
562     FROM videos v
563     JOIN sources s
564         ON s.account_id = v.user_id
565         AND s.provider = 'telegram'
566         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576                                     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 # rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598

```

```

599
600     def _now() -> datetime:
601         """Timezone-aware UTC 'now', used for policy timestamps."""
602         return datetime.now(UTC)
603
604
605     def _row_to_account(row: aiosqlite.Row) -> Account:
606         return Account(
607             id=row["id"],
608             telegram_id=row["telegram_id"],
609             provider=row["provider"],
610             display_name=row["display_name"],
611             created_at=row["created_at"], # stored as ISO string; pydantic parses
612         )
613
614
615     def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616         return TelegramSession(
617             account_id=row["account_id"],
618             phone=row["phone"],
619             session_path=row["session_path"],

```

4. user (2026-07-06T06:08:40.700Z)

```

54:CREATE TABLE IF NOT EXISTS videos (
55-     id            INTEGER PRIMARY KEY AUTOINCREMENT,
56-     user_id       INTEGER NOT NULL REFERENCES accounts(id),
57-     channel_id    INTEGER NOT NULL,
58-     message_id    INTEGER NOT NULL,
59-     document_id   INTEGER,
60-     duration_s    INTEGER,
61-     width         INTEGER,
62-     height        INTEGER,
63-     size_bytes    INTEGER,
64-     mime_type     TEXT,
65-     matched       INTEGER NOT NULL DEFAULT 0,
66-     forwarded     INTEGER NOT NULL DEFAULT 0,
67-     seen_at      TEXT NOT NULL,
68-     UNIQUE (user_id, channel_id, message_id)
69-);
70-
71:CREATE TABLE IF NOT EXISTS filter_config (
72-     user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
73-     config_json   TEXT NOT NULL
74-);
75-""
76-
77-# Note: the ``idx_videos_user_document`` partial unique index is created by
78-# migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
79-# which is present on fresh tables but only added to legacy v1 DBs by the
80-# migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
81-# DBs that don't yet have the column.
82-
83-
84-async def init_db(db_path: str) -> None:
85-    --
274:        CREATE TABLE videos (
275-            id            INTEGER PRIMARY KEY AUTOINCREMENT,
276-            user_id       INTEGER NOT NULL REFERENCES accounts(id),
277-            channel_id    INTEGER NOT NULL,
278-            message_id    INTEGER NOT NULL,
279-            document_id   INTEGER,
280-            duration_s    INTEGER,
281-            width         INTEGER,
282-            height        INTEGER,

```

```

283-         size_bytes  INTEGER,
284-         mime_type   TEXT,
285-         matched     INTEGER NOT NULL DEFAULT 0,
286-         forwarded   INTEGER NOT NULL DEFAULT 0,
287-         seen_at     TEXT NOT NULL,
288-         UNIQUE (user_id, channel_id, message_id)
289-     )
290-     """
291-     "id, user_id, channel_id, message_id, document_id, duration_s, width, "
292-     "height, size_bytes, mime_type, matched, forwarded, seen_at",
293- ),
294- (
295-     "filter_config",
296-     """
297-     CREATE TABLE filter_config (
298-         user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
299-         config_json  TEXT NOT NULL
300-     )
301-     """

```

5. assistant (2026-07-06T06:08:43.325Z)

The finding hinges on whether a v1 DB can hold a `videos` row with no matching `channels` row. Let me verify the v1 insert path and the dedup shims.

6. user (2026-07-06T06:08:46.125Z)

```

        meta.width,
        channel_id,
        message_id,
        existing.channel_id,
        existing.message_id,
    )
    return

# Lazily record the channel (best-effort; failures are non-fatal).
try:
    await db.upsert_channel(
        conn,
        Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
    )
except Exception:
    log.warning(
        "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
    )

result = engine_holder[ENGINE_KEY].evaluate(meta)
record = VideoRecord(
    user_id=user.id,
    channel_id=channel_id,
    message_id=message_id,
    document_id=document_id,
    duration_s=meta.duration_s,
    width=meta.width,
    height=meta.height,
    size_bytes=meta.size_bytes,
    mime_type=meta.mime_type,
    matched=result.matched,
)
try:
    inserted = await db.insert_video(conn, record)
except sqlite3.IntegrityError:
    # Lost a dedup race: a concurrent delivery of the same file
    # (same document_id or same message instance) won the insert.
    # The three-tier check above already classified this as new, so

```

```
# the collision means another task is processing (or has
# processed) it ? log and skip.
log.info(
```

7. user (2026-07-06T06:08:46.952Z)

```
582:async def upsert_channel(conn: aiomysqlite.Connection, channel: Channel) -> Channel:
670:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
```

8. assistant (2026-07-06T06:08:50.620Z)

The finding's premise is confirmed: v1's `upsert\_channel` is best-effort (exception caught + logged, non-fatal), `insert\_video` proceeds regardless, and `videos.channel\_id` has NO foreign key to channels. So an orphan videos row is possible in v1. Now let me verify the dedup shims and run the actual repro.

9. user (2026-07-06T06:08:51.973Z)

```
1121:async def get_video(
1137:async def get_video_by_document_id(
```

10. user (2026-07-06T06:08:54.260Z)

```
1121     async def get_video(
1122         conn: aiomysqlite.Connection, user_id: int, channel_id: int, message_id: int
1123     ) -> VideoRecord | None:
1124         """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126         Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127         :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128         :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129         """
1130         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131         if src is None:
1132             return None
1133         item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134         return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137     async def get_video_by_document_id(
1138         conn: aiomysqlite.Connection, user_id: int, document_id: int
1139     ) -> VideoRecord | None:
1140         """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141         dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143         if item is None:
1144             return None
1145         src = await get_source_by_id(conn, item.source_id)
1146         if src is None:
1147             return None
1148         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151     async def get_video_by_content_fingerprint(
1152         conn: aiomysqlite.Connection,
1153         user_id: int,
1154         size_bytes: int,
1155         duration_s: int,
1156         height: int,
1157         width: int,
1158     ) -> VideoRecord | None:
1159         """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160         dedup_key = compute_dedup_key(
1161             provider="telegram",
```

```

1162         size_bytes=size_bytes,
1163         duration_s=duration_s,
1164         height=height,
1165         width=width,
1166     )
1167     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168     if item is None:
1169         return None
1170     src = await get_source_by_id(conn, item.source_id)
1171     if src is None:
1172         return None
1173     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176 async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) -> Source | None:
1177     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178     conn.row_factory = aiomysqlite.Row
1179     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180         row = await cur.fetchone()
1181     return _row_to_source(row) if row else None
1182
1183
1184 async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),

```

11. assistant (2026-07-06T06:09:00.533Z)

Now I understand the full picture. Let me trace the critical logic carefully.

For an orphan video (channel_id with no channels row):

- Migration backfills `sources` from `channels` only (line 522-524) ? no source row for the orphan channel.
- Migration backfills `content\_items` via INNER JOIN sources (line 562-566) ? no match, so no content_item.

But there's a subtlety I must check: even if the migration drops the content_item, does the next scan actually re-forward? The dedup check happens *before* insert. `get\_video` at line 1130 resolves source by provider_id ? but on the *next scan*, `insert\_video`'s shim (line 1192) calls `upsert\_source`, which would re-create the source. But the dedup check runs first. Let me run the actual repro to confirm the end-to-end claim.

12. user (2026-07-06T06:09:03.367Z)

```

670:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
679:    """INSERT INTO videos
---forwarded col in content_items?---
72:    forwarded    INTEGER NOT NULL DEFAULT 0,
455:    forwarded    INTEGER NOT NULL DEFAULT 0,
461:    "height, size_bytes, mime_type, matched, forwarded, seen_at",
673:    forwarded=bool(row["forwarded"]),

```

```

1232:async def mark_forwarded(conn: aioredis.Connection, video_id: int) -> None:
1233:    """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1242:    # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1252:    ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1269:        forwarded=False,
1337:        forwarded=False,

```

13. assistant (2026-07-06T06:09:07.281Z)

Now let me run the actual repro to verify the end-to-end claim.

14. assistant (2026-07-06T06:09:18.719Z)

```

import asyncio, aioredis
from tg_video_filter import db

async def main():
    conn = await aioredis.connect(":memory:")
    conn.row_factory = aioredis.Row
    # Build v1 legacy schema (pre-ADR-0011): accounts + channels + videos + filter_config
    await conn.executescript("""
CREATE TABLE accounts (id INTEGER PRIMARY KEY AUTOINCREMENT, telegram_id INTEGER, provider
TEXT, display_name TEXT, created_at TEXT);
CREATE TABLE channels (user_id INTEGER, channel_id INTEGER, title TEXT, seen_first_at TEXT,
PRIMARY KEY(user_id, channel_id));
CREATE TABLE videos (
    id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL, channel_id INTEGER NOT
NULL,
    message_id INTEGER NOT NULL, document_id INTEGER, duration_s INTEGER, width INTEGER,
height INTEGER,
    size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0, forwarded INTEGER
NOT NULL DEFAULT 0,
    seen_at TEXT NOT NULL, UNIQUE(user_id, channel_id, message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY, config_json TEXT NOT NULL);
""")
    await conn.execute("INSERT INTO accounts (id, telegram_id, provider, display_name,
created_at) VALUES (1, 999, 'telegram', 'u', '2020-01-01')")
    # Orphan video: channel_id=-1005 has NO channels row
    await conn.execute("INSERT INTO videos (user_id, channel_id, message_id, document_id,
matched, forwarded, seen_at) VALUES (1, -1005, 77, 888, 1, 1, '2020-01-01')")
    await conn.commit()

    # Now apply the ADR-0011 schema + migration 003 the way init_db would.
    await db._apply_schema(conn)
    await conn.commit()

    # inspect
    async with conn.execute("SELECT count(*) c FROM videos_deprecated") as cur:
        print("videos_deprecated rows =", (await cur.fetchone())["c"])
    async with conn.execute("SELECT count(*) c FROM content_items") as cur:
        print("content_items rows =", (await cur.fetchone())["c"])
    async with conn.execute("SELECT count(*) c FROM sources") as cur:
        print("sources rows =", (await cur.fetchone())["c"])

    v1 = await db.get_video(conn, 1, -1005, 77)
    print("get_video(1,-1005,77) =", v1)
    v2 = await db.get_video_by_document_id(conn, 1, 888)
    print("get_video_by_document_id(1,888) =", v2)

    await conn.close()

asyncio.run(main())

```

15. user (2026-07-06T06:09:18.744Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro8.py (file state is current in your context ? no need to Read it back)

16. assistant (2026-07-06T06:09:22.542Z)

I need to check how `\_apply\_schema` invokes migration 003 ? whether it runs all migrations. Let me check the schema application flow.

17. user (2026-07-06T06:09:24.751Z)

```
33:_SCHEMA = ""
175:# migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
177:# migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
181:async def init_db(db_path: str) -> None:
188:     await _apply_schema(conn)
191:async def _apply_schema(conn: aiosqlite.Connection) -> None:
197:     await conn.executescript(_SCHEMA)
198:     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
215:# Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
369:     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
391:     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
502:     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
509:     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510:     # dropped by _apply_schema post-003). Nothing to backfill.
```

18. user (2026-07-06T06:09:27.312Z)

```
191     async def _apply_schema(conn: aiosqlite.Connection) -> None:
192         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193         # and a busy timeout so concurrent access waits instead of failing immediately.
194         await conn.execute("PRAGMA journal_mode = WAL")
195         await conn.execute("PRAGMA foreign_keys = ON")
196         await conn.execute("PRAGMA busy_timeout = 5000")
197         await conn.executescript(_SCHEMA)
198         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202         # any *empty* recreations of those legacy tables from a subsequent
203         # ``init_db`` are dropped here so they don't shadow the renamed copies.
204         if await _table_exists(conn, "channels_deprecated"):
205             await conn.execute("DROP TABLE IF EXISTS channels")
206             await conn.execute("DROP TABLE IF EXISTS videos")
207             await conn.execute("DROP TABLE IF EXISTS filter_config")
208         await _apply_migrations(conn)
209         await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
```

19. assistant (2026-07-06T06:09:31.514Z)

`\_apply\_schema` runs `executescript(\_SCHEMA)` (which uses `CREATE TABLE IF NOT EXISTS`, so it won't clobber my pre-seeded legacy tables) then `\_apply\_migrations`. But my repro's legacy tables lack the exact columns/DDL `\_SCHEMA` expects. Since `\_SCHEMA` uses `IF NOT EXISTS`, my

seeded tables stay. However `\_apply\_migrations` may run migrations 001/002 which alter the legacy tables. Let me just run the repro.

20. user (2026-07-06T06:09:36.542Z)

```
videos_deprecated rows = 1
content_items rows = 0
sources rows = 0
get_video(1,-1005,77) = None
get_video_by_document_id(1,888) = None
```

21. assistant (2026-07-06T06:09:44.567Z)

The repro confirms the mechanism end-to-end:

- 1 orphan video in `videos\_deprecated`
- 0 content_items (INNER JOIN dropped it)
- Both dedup shims return `None` ? the video is invisible post-migration.

On the next scan, `insert\_video` (line 1192) will `upsert\_source` and re-insert the content_item, treating it as new ? duplicate forward. The finding's core claim is correct.

Now let me adversarially test the trigger's realism. The key question: ****can an orphan videos row genuinely exist in a real v1 DB?**** The finding relies on `upsert\_channel` failing while `insert\_video` succeeds. Let me examine whether `upsert\_channel` failure is realistic and whether there are other orphan paths.

22. user (2026-07-06T06:09:46.408Z)

```
async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
    """Insert a (user, channel) row, or no-op if it already exists."""
    await conn.execute(
        """INSERT INTO channels (user_id, channel_id, title, seen_first_at)
        VALUES (?, ?, ?, ?)
        ON CONFLICT(user_id, channel_id) DO UPDATE SET title=excluded.title""",
        (
            channel.user_id,
            channel.channel_id,
            channel.title,
            channel.seen_first_at.isoformat(),
        ),
    )
    await conn.commit()
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        "SELECT * FROM channels WHERE user_id = ? AND channel_id = ?",
        (channel.user_id, channel.channel_id),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_channel(row) # type: ignore[return-value]
```

--- video repos -----

```
async def get_video(
    conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
) -> VideoRecord | None:
    """Look up a video by its dedup key. Returns None if not seen before."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
        WHERE user_id = ? AND channel_id = ? AND message_id = ?""",
        (user_id, channel_id, message_id),
    ) as cur:
```

```

        row = await cur.fetchone()
        return _row_to_video(row) if row else None

async def get_video_by_document_id(
    conn: aiosqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:
    """Tier-1 content dedup: same Telegram file id ? same video.

    Telegram reuses ``document.id`` when the same file is forwarded across
    channels, so this catches cross-channel duplicates at zero cost.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
        WHERE user_id = ? AND document_id = ?""",
        (user_id, document_id),
    ) as cur:
        row = await cur.fetchone()
        return _row_to_video(row) if row else None

async def get_video_by_content_fingerprint(

```

23. assistant (2026-07-06T06:10:00.744Z)

Now I have the full picture on trigger realism. Let me assess honestly:

****How does an orphan row occur in v1?**** The ``upsert_channel`` call is wrapped in try/except and non-fatal (telethon_client.py:238-247), and ``insert_video`` proceeds regardless. ``videos.channel_id`` has NO FK to channels. So the sequence: channel upsert raises ? caught+logged ? `insert_video` still runs ? orphan videos row with no channels row.

Realistic triggers for ``upsert_channel`` to fail while ``insert_video`` succeeds:
 - ``_channel_title(event)`` raising, or a transient DB error mid-commit on the channels insert, or the ``seen_first_at.isoformat()`` path. These are narrow but genuinely possible ? the code author deliberately made it best-effort **because** they anticipated failures. If failure were impossible, the try/except would be dead code.

Also note: even a **successful** channel upsert followed by a channel deletion, or a v1 DB where a channels row was pruned but videos retained, produces the same orphan. And crucially, ``videos.title`` comes from ``channels``, but the migration's ``sources`` backfill (line 522) also requires a channels row.

So the trigger is real but narrow ? it requires the best-effort channel upsert to have failed at least once in the DB's history for a video that was matched+forwarded. That is a genuine latent data-corruption path (silent duplicate re-delivery + loss of matched/forwarded state), consistent with ****medium**** severity. It's not a blocker (requires a specific pre-existing DB anomaly, not the common path), and not merely low (when triggered it causes real duplicate delivery + state loss, a data-correctness defect).

Let me confirm the migration code (line 562's INNER JOIN) is indeed new in this PR's diff and not pre-existing.

24. user (2026-07-06T06:10:02.222Z)

(Bash completed with no output)

25. assistant (2026-07-06T06:10:09.031Z)

Confirmed: ``content_items``, the ``JOIN sources`` backfill, and migration 003 do not exist on ``main`` ? this is entirely new code shipping in this PR's diff.

The finding is verified end-to-end:

- The INNER JOIN at db.py:562-566 drops any videos row whose channel_id has no channels row.
- v1 can legitimately produce such orphan rows (best-effort non-fatal channel upsert + no FK + insert_video proceeds).
- Repro confirms: orphan video ? 0 content_items, both dedup shims return None ? video invisible post-migration ? re-ingested and re-forwarded on next scan (duplicate delivery), with matched/forwarded state silently lost.
- This code is new in this PR's diff, not pre-existing.

Severity stays **medium**: the failure requires a pre-existing orphan-video anomaly in the legacy DB (a narrow, non-default condition), but when triggered it causes real duplicate delivery to the user plus loss of dedup state ? a data-correctness defect, appropriately medium rather than blocker.

26. user (2026-07-06T06:10:18.630Z)

Structured output provided successfully